1. What is the default value assigned to array elements in C#?
It's the default value of the data type itself. For `int[]` all elements will be 0, for `bool[]` they'll be `false`, and for arrays of objects or strings they'll be `null`.

2. What is the difference between Array.Clone() and Array.Copy()?
`Clone()` creates a completely new copy of the array and returns it as an `object` (so you need to cast it). `Array.Copy()`, on the other hand, copies elements from a source array into an existing destination array, and gives you more flexibility like specifying the number of elements and the starting index. Both, however, perform a shallow copy — not a true deep copy — if the elements inside the array are reference types.

3. What is the difference between GetLength() and Length for multidimensional arrays?
`Length` returns the total count of all elements in the array (rows × columns combined). `GetLength(dimension)`, however, returns the number of elements in a specific dimension only (e.g., `GetLength(0)` for rows and `GetLength(1)` for columns).

4. What is the difference between Array.Copy() and Array.ConstrainedCopy()?
If `Array.Copy()` fails midway through copying, it can leave the destination array in a partially modified state. `Array.ConstrainedCopy()`, on the other hand, guarantees atomicity — if the operation fails for any reason, the destination array is rolled back to its original state as if nothing happened.

5. Why is foreach preferred for read-only operations on arrays?
Because it's simpler and cleaner to read (no need to deal with indices), and it reduces the chance of errors like off-by-one mistakes. However, it's read-only in the sense that you can't directly modify an element's value through it, which makes it perfect when your goal is just reading, not modifying.

6. Why is input validation important when working with user inputs?
It prevents the program from crashing if the user enters something invalid (like text instead of a number), and ensures the data entering the program is actually correct and makes sense before being processed. This protects the program from unexpected exceptions and incorrect results.

7. How can you format the output of a 2D array for better readability?
You can use `\t` (tab), or `String.Format`/`PadLeft`/`PadRight` to align columns properly, or use a loop that prints separators (like `---` lines) between rows to make it look more like an actual table.

8. When should you prefer a switch statement over if-else?

When you have a single variable being compared against specific fixed values (equality checks), especially when there are many conditions. `switch` tends to be cleaner and easier to read and maintain. But if the conditions are complex or involve comparisons like > or &&, `if-else` is more suitable.

9. What is the time complexity of Array.Sort()?

It uses Introspective Sort (a hybrid of Quicksort, Heapsort, and Insertion sort), with an average time complexity of O(n log n), and it stays around O(n log n) even in the worst case thanks to the fallback to Heapsort.

10. Which loop (for or foreach) is more efficient for calculating the sum of an array, and why?

Generally, a `for` loop tends to be slightly faster with arrays, especially with value types, because it accesses elements directly by index without the overhead of creating an enumerator. However, the performance difference is usually very small and only noticeable with very large arrays, so in practice both are suitable and the difference won't matter much for most typical applications.